

《计算机系统基础》

第3章 程序的机器级表示 I：基础

授课老师:

何璐璐

luluhe@whu.edu.cn

本节内容

- Intel 处理器的历史与架构
- C 代码，汇编代码，机器级代码
- 汇编语言基础：寄存器、操作数、move数据传送指令
- 算术和逻辑操作

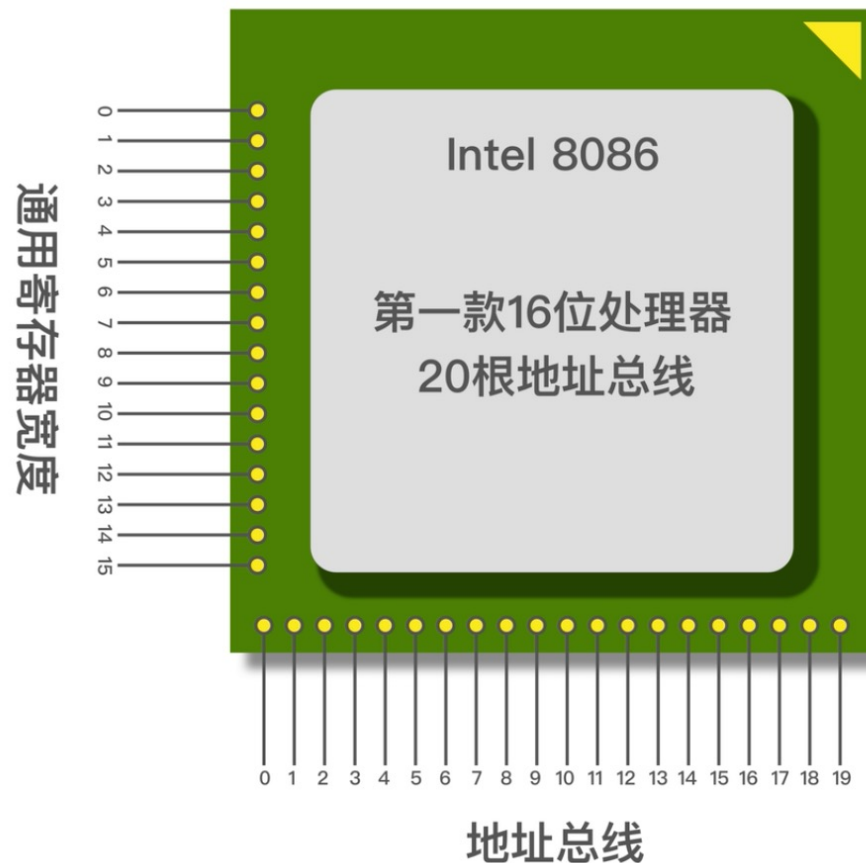
Intel x86 处理器

- **统治笔记本、台式电脑和服务市场**
- **不断进化的设计**
 - 向后兼容的特性（可以兼容到1978年设计的 8086）
 - 随着时代的发展不断增加新的特性

Intel x86 演化过程中的里程碑

名称	日期	晶体管数量	频率 (MHz)
■ 8086	1978	29K	5-10

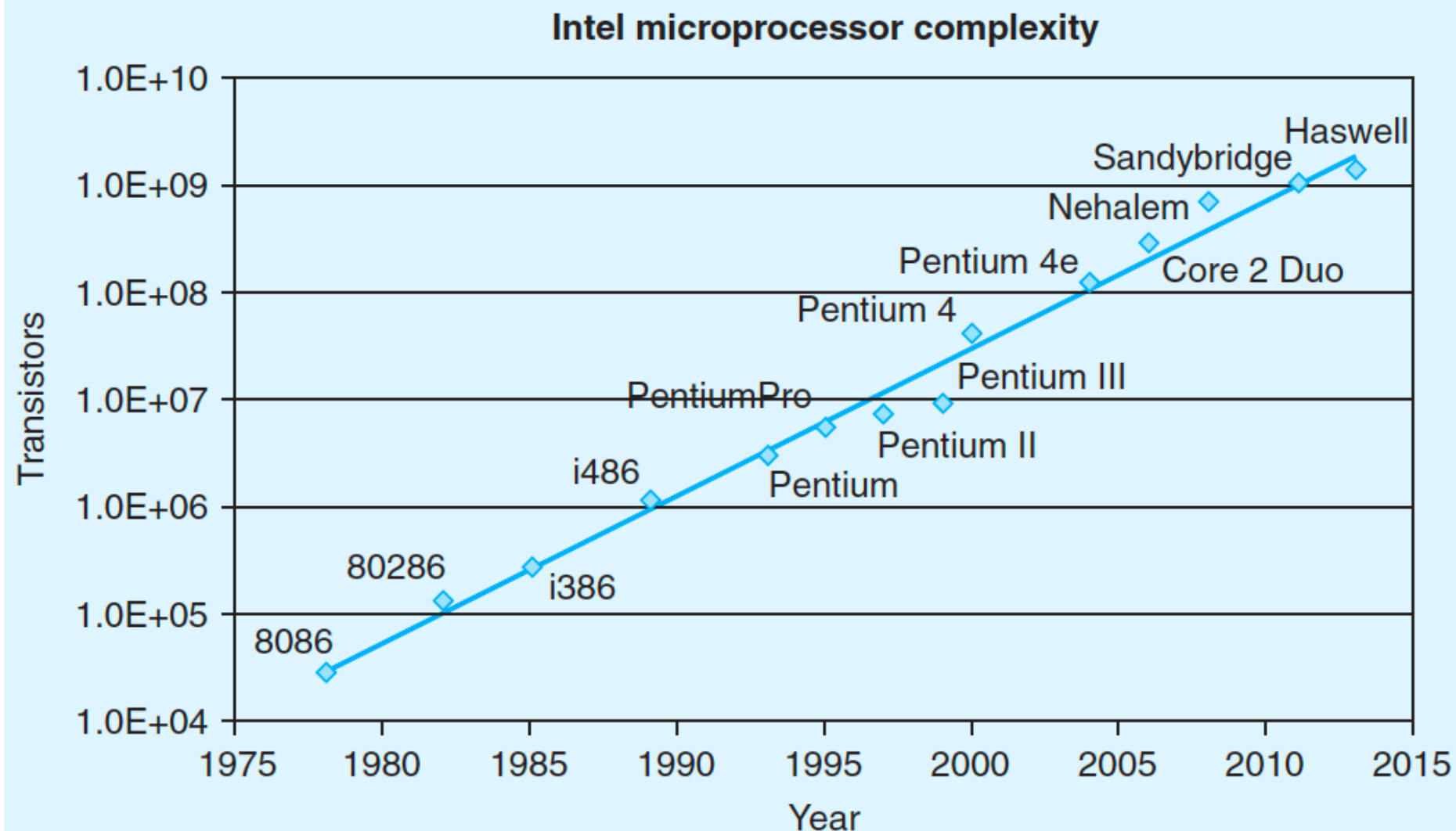
- 第一代16位 Intel 微处理器， IBM PC & MS-DOS 操作系统的基础
- 1MB 地址空间



Intel x86 演化过程中的里程碑

名称	日期	晶体管数量	频率 (MHz)
■ 8086	1978	29K	5-10
▪ 第一代16位 Intel 微处理器， IBM PC & MS-DOS 操作系统的基础 ▪ 1MB 地址空间			
■ 386	1985	275K	16-33
▪ 第一台32位 Intel 微处理器，被称称 IA32 ▪ 增加了“平面寻址模式flat-addressing”，支持 Unix 操作系统			
■ Pentium 4E	2004	125M	2800-3800
▪ 第一台64位 Intel x86 处理器，被称为 x86-64			
■ Core 2	2006	291M	1060-3333
▪ Intel 的第一个多核微处理器			
■ Core i7	2008	731M	1600-4400
▪ 四核			

摩尔定律



前沿技术 2022.8.20

■ Intel 13th Gen Meteor Lake处理器

■ 桌面型号

- P核达到6.0GHz，E核达到4.3GHz
- 可以有8个P核，16个E核
- 36MB L3高速缓存



Intel x86 处理器

■ x86 是复杂指令集计算机架构 (CISC)

- 有许多不同的指令和许多不同的形式
 - Linux仅仅用到了一小部分

■ 比较：精简指令集计算机架构 (RISC)

- RISC: 指令很少，每个指令的模式很少
- RISC 的速度可以非常迅速
- 然而Intel的速度依然领先！
 - 在速度方面是做到了，但是在低功耗方面就做得没那么成功

x86领域另一个巨头：AMD

■ 一开始.....

- AMD紧随Intel之后
- 速度慢一点，价格便宜很多

■ 之后.....

- 从DEC公司和其他走下坡路的公司里招募了许多顶级电路设计师
- 打造Opteron 处理器：Pentium 4的劲敌
- 发展x86-64，AMD 在64位领域的自主尝试

■ 近些年.....

- Intel采取行动
 - 引领世界半导体技术发展
- AMD暂时落后
 - 依赖外部的半导体制造商
- 卷土重来？

Intel 64位的历史

- **2001: Intel 从IA32 到IA64 的激进尝试**
 - 完全不同的架构 (Itanium 安腾)
- **IA32 的代码作为历史遗留执行**
 - 然而表现不尽人意
- **2003: AMD 开始发展自己的解决方案**
 - x86-64 (现在称作 “AMD64”)
- **Intel 察觉到，应该着重研发IA64**
 - 不想承认自己的错误，或者AMD更为优秀
- **2004: Intel 宣布IA32 的EM64T 扩展**
 - 扩展内存至64位技术
 - 几乎与x86-64 一模一样！
- **除了部分较为低廉的x86处理器外，大都支持x86-64**
 - 但是，仍有许多代码以32位模式运行



2022 Apple M2

Apple M2是苹果公司设计的一系列基于ARM 架构的片上系统(SoC)，作为其Mac 台式机、笔记本电脑和iPadPro 平板电脑的CPU 和GPU。



本课程选用教材的内容涵盖

■ IA32

- 传统的x86架构

■ x86-64

- 现行的标准
- `linux> gcc hello.c`
- `linux> gcc -m64 hello.c`

■ 呈现形式

- 书中主体内容围绕x86-64 展开
- 网络旁注中涉及IA32
- 我们只对x86-64 作讲解

本节内容

- Intel 处理器的历史与架构
- **C 代码，汇编代码，机器级代码**
- 汇编语言基础：寄存器、操作数、move数据传送指令
- 算术和逻辑操作

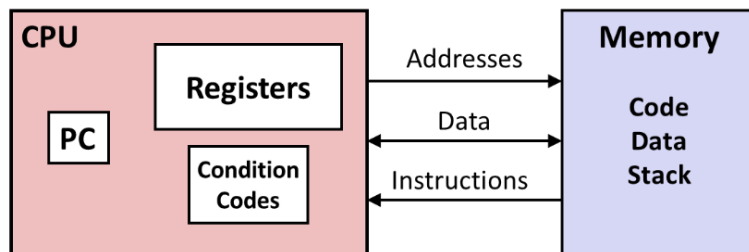
抽象层次

C 程序

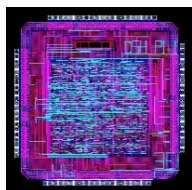
```
#include <stdio.h>
int main(){
    int i, n = 10, t1 = 0, t2 = 1, nxt;
    for (i = 1; i <= n; ++i){
        printf("%d, ", t1);
        nxt = t1 + t2;
        t1 = t2;
        t2 = nxt; }
    return 0; }
```

这是漂亮干净的一层，
但要小心...

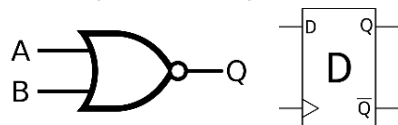
汇编程序



计算机设计



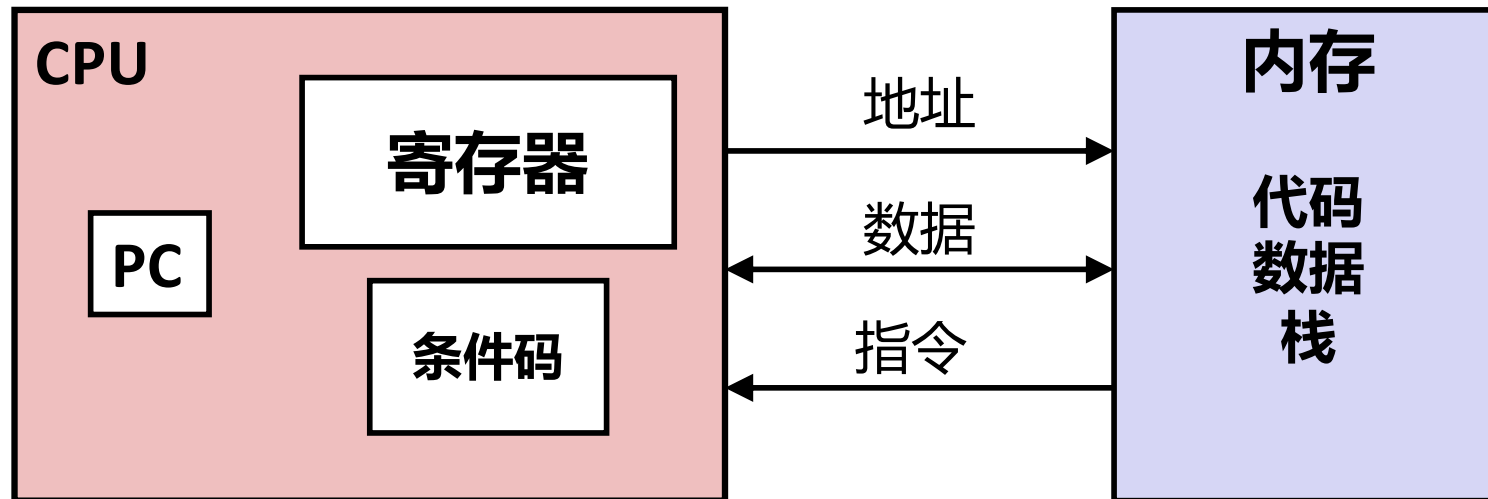
Gates, clocks, circuit layout, ...



一些定义

- **架构**（指令集体系结构，ISA）：理解或书写汇编代码、机器级代码应该掌握的处理器设计的知识
 - 例如：指令集的规范、寄存器
- **微架构**：架构的实现
 - 例如：高速缓存的大小、核心频率
- 代码格式：
 - **机器码**：处理器执行的字节级程序
 - **汇编代码**：机器代码的文本表示
- ISAs 举例：
 - Intel: x86, IA32, Itanium, x86-64
 - ARM: Apple M1/M2, 几乎用于所有手机
 - MIPS , RISC-V

汇编/机器代码视图



一个程序员应该知道：

■ PC: 程序计数器

- 下一条指令的地址
- 被叫做“RIP” (x86-64)

■ 寄存器文件

- 程序频繁使用的数据

■ 条件码

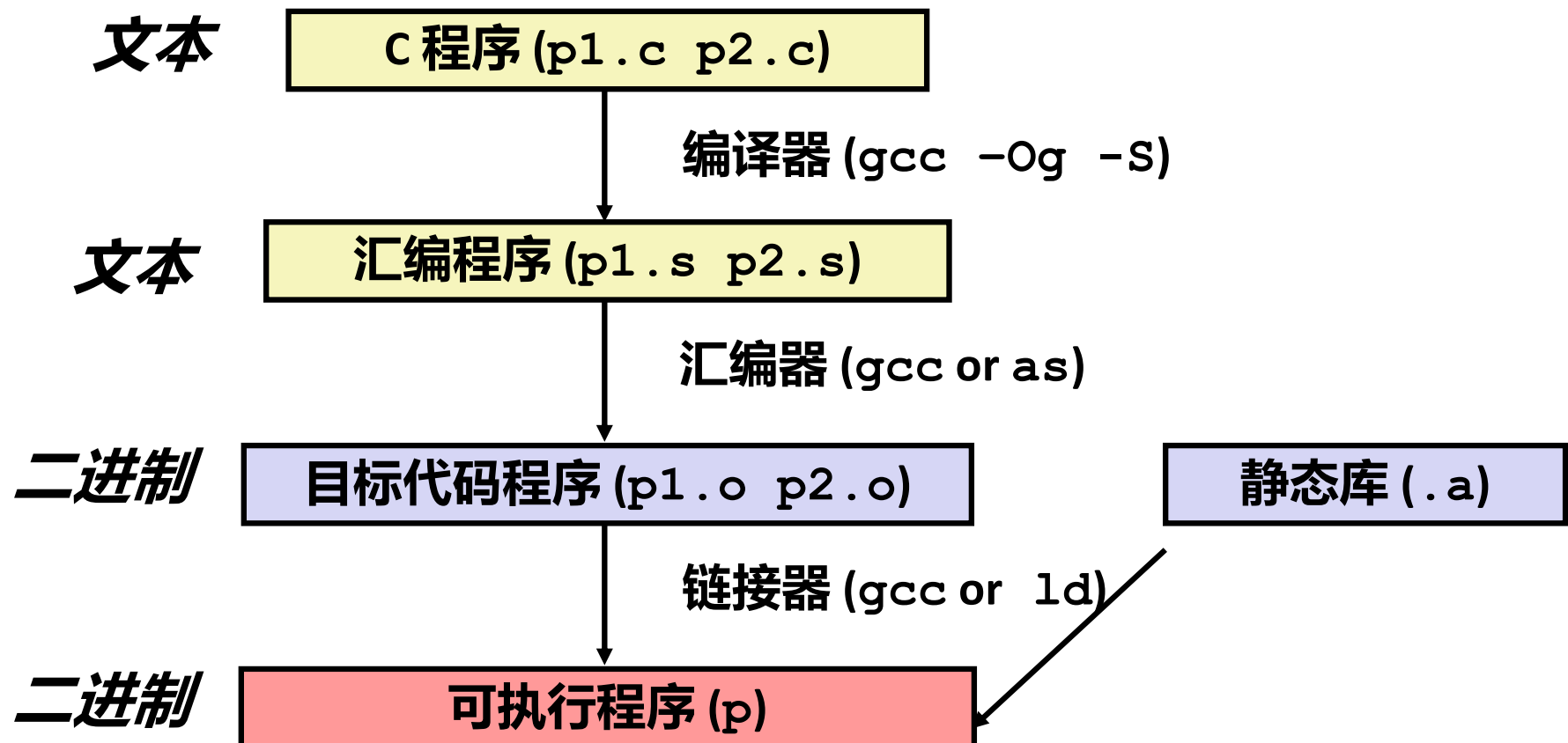
- 储存最近一次算术或逻辑运算的结果
- 用于条件分支语句

■ 内存

- 字节可寻址数组
- 代码和用户数据
- 为程序创建栈帧

C 文件是如何转变为可执行程序

- 代码文件 `p1.c` `p2.c`
- 使用如下命令来编译：`gcc -Og p1.c p2.c -o p`
 - 使用最基本的优化选项(`-Og`)
 - 将生成的二进制文件放入文件`p`中



编译生成汇编语言

C 代码 (sum.c)

```
long plus(long x, long y);

void sumstore(long x, long y,
              long *dest)
{
    long t = plus(x, y);
    *dest = t;
}
```

生成的 x86-64 汇编语言文件

```
sumstore:
    pushq    %rbx
    movq     %rdx, %rbx
    call     plus
    movq     %rax, (%rbx)
    popq     %rbx
    ret
```

使用命令来获取

```
gcc -Og -S sum.c
```

生成文件sum.s

警告：由于不同版本的 gcc 和不同的编译器设置，在不同机器(Linux、Mac OS-X 等)上会得到非常不同的结果。

汇编代码真实的样子

```
        .globl  sumstore
        .type   sumstore, @function
sumstore:
.LFB35:
        .cfi_startproc
        pushq   %rbx
        .cfi_def_cfa_offset 16
        .cfi_offset 3, -16
        movq    %rdx, %rbx
        call    plus
        movq    %rax, (%rbx)
        popq    %rbx
        .cfi_def_cfa_offset 8
        ret
        .cfi_endproc
.LFE35:
        .size   sumstore, .-sumstore
```

汇编代码真实的样子

看起来很奇怪并以“.”开头的语句通常是伪指令

```
.globl  sumstore
.type   sumstore, @function

sumstore:
.LFB35:

    .cfi_startproc
pushq   %rbx
.cfi_def_cfa_offset 16
.cfi_offset 3, -16
movq    %rdx, %rbx
call    plus
movq    %rax, (%rbx)
popq    %rbx
.cfi_def_cfa_offset 8
ret
.cfi_endproc

.LFE35:

.size   sumstore, .-sumstore
```

```
sumstore:
    pushq   %rbx
    movq    %rdx, %rbx
    call    plus
    movq    %rax, (%rbx)
    popq    %rbx
    ret
```

汇编特性：数据类型

- **1、2、4 或 8 字节的“整形”数据**
 - 数据值
 - 地址（无类型指针）
- **4、8 或 10 字节的浮点型数据**
- **代码：指令的字节序列编码**
- **没有像数组或结构体这样的聚合类**
 - 只是在内存中分配连续的字节

汇编语言特点：操作

- **对存储在寄存器或内存上的数据进行算术操作**
- **在内存和寄存器之间传送数据**
 - 从内存向寄存器加载数据
 - 向内存存储寄存器的数据
- **控制的转移**
 - 无条件跳转到过程或从过程返回
 - 条件分支语句

目标代码

sumstore 的代码

```
gcc -Og -c sum.c
```

0x0400595:

0x53

0x48

0x89

0xd3

0xe8

0xf2

0xff

0xff

0xff

0x48

0x89

0x03

0x5b

0xc3

- 共 14 个字节
- 每条指令为 1、3 或 5 个字节
- 从地址 0x0400595 开始

■ 汇编器

- 将 .s 文件编译为成 .o 文件
- 每条指令的二进制编码
- 几乎完整的可执行代码
- 还缺少不同文件之间的链接

■ 链接器

- 解析文件之间的引用
- 与静态运行库结合
 - 例如 , malloc, printf 的代码
- 一些库是动态链接的
 - 链接发生在程序开始执行时

机器指令示例

```
*dest = t;
```

```
movq %rax, (%rbx)
```

```
0x40059e: 48 89 03
```

■ C 代码

- 将t 的值存放在dest 指向的地方

■ 汇编语言

- 将 8 字节值移动到内存
 - x86-64 中的四字
- 操作数:
 - t: 寄存器 %rax
 - dest: 寄存器 %rbx
 - *dest: 内存 M[%rbx]

■ 目标代码

- 3字节指令
- 存储在地址0x40059e处

反汇编目标代码

反汇编后

```
0000000000400595 <sumstore>:
400595: 53                push    %rbx
400596: 48 89 d3          mov     %rdx,%rbx
400599: e8 f2 ff ff ff    callq   400590 <plus>
40059e: 48 89 03          mov     %rax, (%rbx)
4005a1: 5b                pop     %rbx
4005a2: c3                retq
```

■ 反汇编器

`objdump -d sum`

- 检查目标代码的有用工具
- 分析指令序列的位模式
- 得到目标代码大致对应的汇编代码
- 可被用于 .out (完全可执行文件) 和 .o 文件

另一种反汇编的方式

目标代码

0x0400595:

0x53

0x48

0x89

0xd3

0xe8

0xf2

0xff

0xff

0xff

0x48

0x89

0x03

0x5b

0xc3

反汇编结果

Dump of assembler code for function sumstore:

0x0000000000400595 <+0>: push %rbx

0x0000000000400596 <+1>: mov %rdx,%rbx

0x0000000000400599 <+4>: callq 0x400590 <plus>

0x000000000040059e <+9>: mov %rax, (%rbx)

0x00000000004005a1 <+12>: pop %rbx

0x00000000004005a2 <+13>: retq

■ 在gdb调试器中

■ 反汇编程序

gdb sum

disassemble sumstore

■ 反汇编过程

x/14xb sumstore

■ 检查从sumstore开始的14个字节

哪些内容可以被反汇编

```
% objdump -d WINWORD.EXE
```

```
WINWORD.EXE:      file format pei-i386
```

```
No symbols in "WINWORD.EXE".
```

```
Disassembly of section .text:
```

```
30001000 <.text>:
```

```
30001000:
```

```
30001001:
```

```
30001003:
```

```
30001005:
```

```
3000100a:
```

**微软最终用户许可协议
禁止逆向工程**

- 任何可以解释为可执行代码的文件
- 反汇编程序检查字节内容并重构汇编代码

本节内容

- Intel 处理器的历史与架构
- C 代码，汇编代码，机器级代码
- **汇编语言基础：寄存器、操作数、move数据传送指令**
- 算术和逻辑操作

x86-64 整数寄存器

%rax	%eax
%rbx	%ebx
%rcx	%ecx
%rdx	%edx
%rsi	%esi
%rdi	%edi
%rsp	%esp
%rbp	%ebp

%r8	%r8d
%r9	%r9d
%r10	%r10d
%r11	%r11d
%r12	%r12d
%r13	%r13d
%r14	%r14d
%r15	%r15d

- 也可以只引用低位 1,2或4 字节
- 不是内存（或缓存）的一部分

一些历史：IA32 寄存器

起源
(大部分已淘汰)

通用目的

%eax	%ax	%ah	%al
%ecx	%cx	%ch	%cl
%edx	%dx	%dh	%dl
%ebx	%bx	%bh	%bl
%esi	%si		
%edi	%di		
%esp	%sp		
%ebp	%bp		

累加 *accumulate*

计数器 *counter*

数据 *data*

基址 *base*

原索引 *source index*

目的索引 *destination index*

栈指针 *stack pointer*

基址指针 *Base pointer*

16 位虚拟寄存器
(向后兼容)

数据传送

■ 数据传送

`movq Source, Dest`

■ 操作数类型

- **立即数**：常整型数据
 - 示例: `$0x400`, `$-533`
 - 类似C常量, 但带有前缀 `'$'`
 - 编码为 1, 2, 或 4 字节
- **寄存器**：16 个整数寄存器之一
 - 示例: `%rax`, `%r13`
 - 但是 `%rsp` 保留为特殊用途
 - 其他的寄存器对特定指令有特殊用途
- **内存**：内存中从寄存器指向地址开始的 8 个连续字节的
 - 示例：`(%rax)`
 - 各类“寻址模式”

`%rax`

`%rcx`

`%rdx`

`%rbx`

`%rsi`

`%rdi`

`%rsp`

`%rbp`

`%rN`

movq 操作数组合示例

	Source	Dest	Src, Dest	C Analog
movq	立即数	Reg	movq \$0x4, %rax	temp = 0x4;
		Mem	movq \$-147, (%rax)	*p = -147;
	寄存器	Reg	movq %rax, %rdx	temp2 = temp1;
		Mem	movq %rax, (%rdx)	*p = temp;
	内存	Reg	movq (%rax), %rdx	temp = *p;

单个指令无法实现数据的内存到内存传送

简单寻址模式

■ 通常 (R) Mem[Reg[R]]

- 寄存器 R 表示内存地址
- 类似于 C 语言中的指针解引用

```
movq (%rcx), %rax
```

■ 偏移量 D(R) Mem[Reg[R]+D]

- 寄存器 R 表示一段内存区间的起始地址
- 常量偏移量 D 表示偏移量的大小

```
movq 8(%rbp), %rdx
```

简单寻址模式示例

```
void  
whatAmI (<type> a, <type> b)  
{  
    ???  
}
```

`%rdi`

`%rsi`

`whatAmI:`

```
    movq    (%rdi), %rax  
    movq    (%rsi), %rdx  
    movq    %rdx, (%rdi)  
    movq    %rax, (%rsi)  
    ret
```

简单寻址模式示例

```
void swap
(long *xp, long *yp)
{
    long t0 = *xp;
    long t1 = *yp;
    *xp = t1;
    *yp = t0;
}
```

swap:

```
movq    (%rdi), %rax
movq    (%rsi), %rdx
movq    %rdx, (%rdi)
movq    %rax, (%rsi)
ret
```

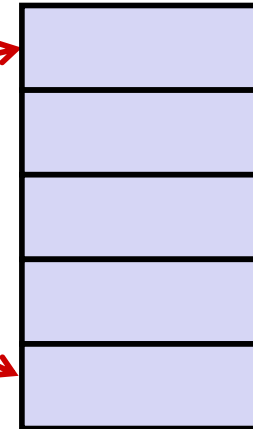
理解 Swap()

```
void swap
(long *xp, long *yp)
{
    long t0 = *xp;
    long t1 = *yp;
    *xp = t1;
    *yp = t0;
}
```

寄存器

%rdi	
%rsi	
%rax	
%rdx	

内存



寄存器	值
%rdi	xp
%rsi	yp
%rax	t0
%rdx	t1

swap:

```
movq    (%rdi), %rax    # t0 = *xp
movq    (%rsi), %rdx    # t1 = *yp
movq    %rdx, (%rdi)    # *xp = t1
movq    %rax, (%rsi)    # *yp = t0
ret
```

理解 Swap()

寄存器

%rdi	0x120
%rsi	0x100
%rax	
%rdx	

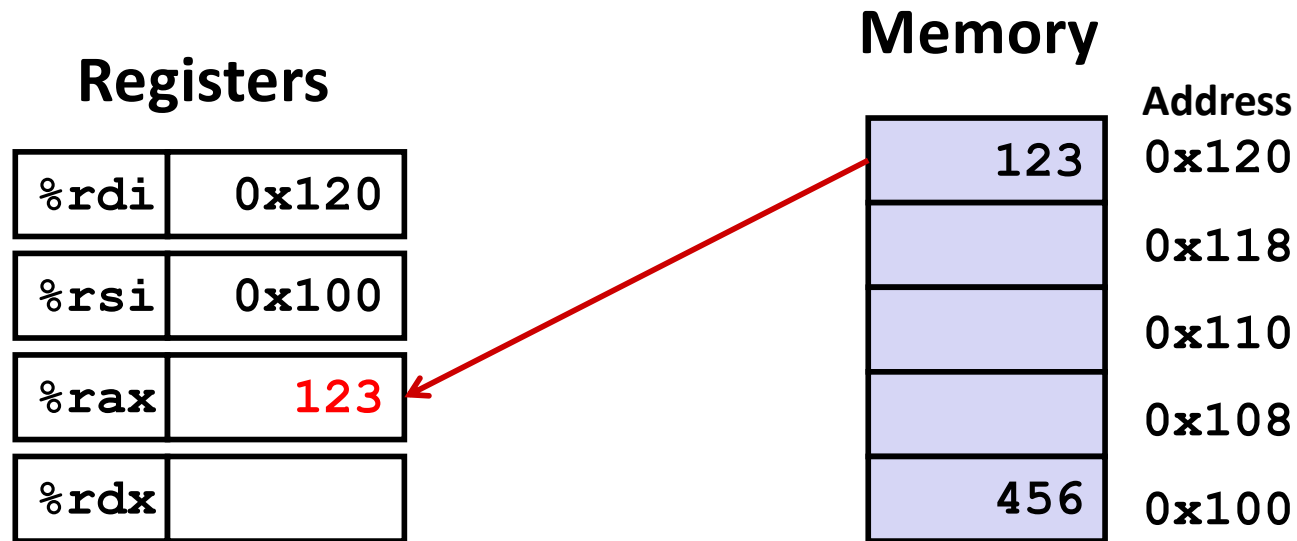
内存

地址	
0x120	123
0x118	
0x110	
0x108	
0x100	456

swap:

```
movq    (%rdi), %rax    # t0 = *xp
movq    (%rsi), %rdx    # t1 = *yp
movq    %rdx, (%rdi)    # *xp = t1
movq    %rax, (%rsi)    # *yp = t0
ret
```

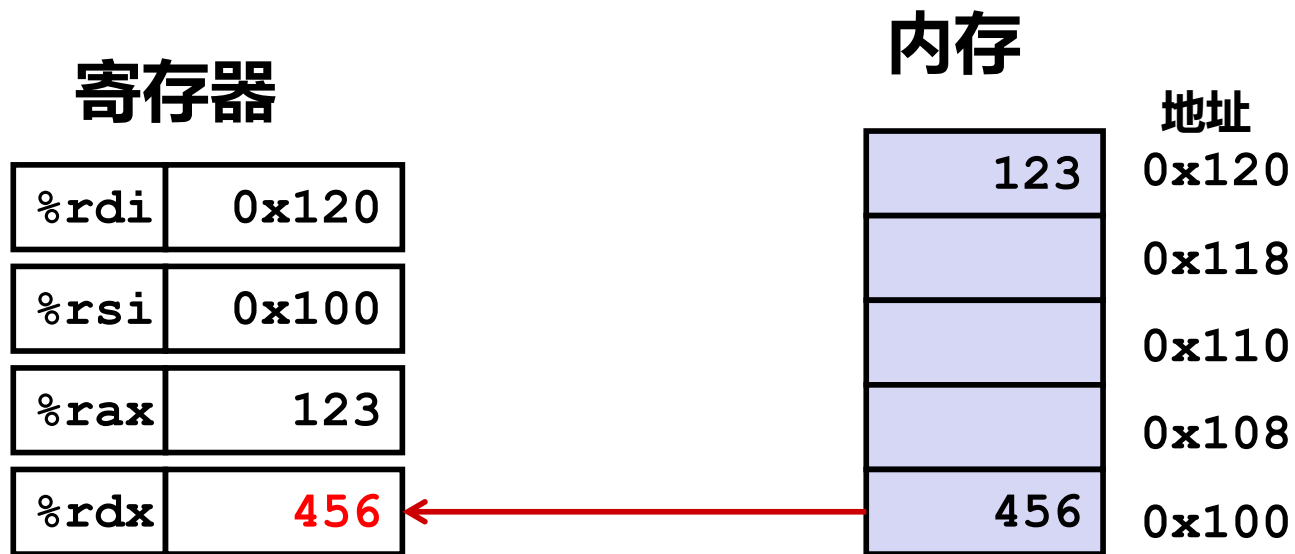
理解Understanding Swap()



swap:

```
movq    (%rdi), %rax    # t0 = *xp
movq    (%rsi), %rdx    # t1 = *yp
movq    %rdx, (%rdi)    # *xp = t1
movq    %rax, (%rsi)    # *yp = t0
ret
```

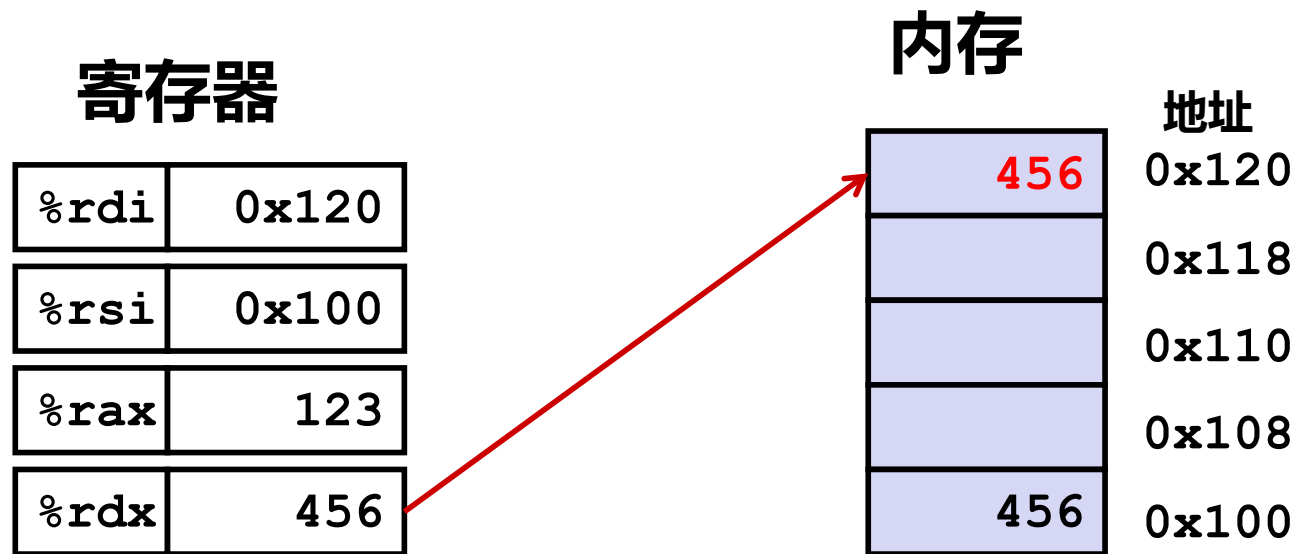
理解 Swap()



swap:

```
movq    (%rdi), %rax    # t0 = *xp
movq    (%rsi), %rdx    # t1 = *yp
movq    %rdx, (%rdi)    # *xp = t1
movq    %rax, (%rsi)    # *yp = t0
ret
```

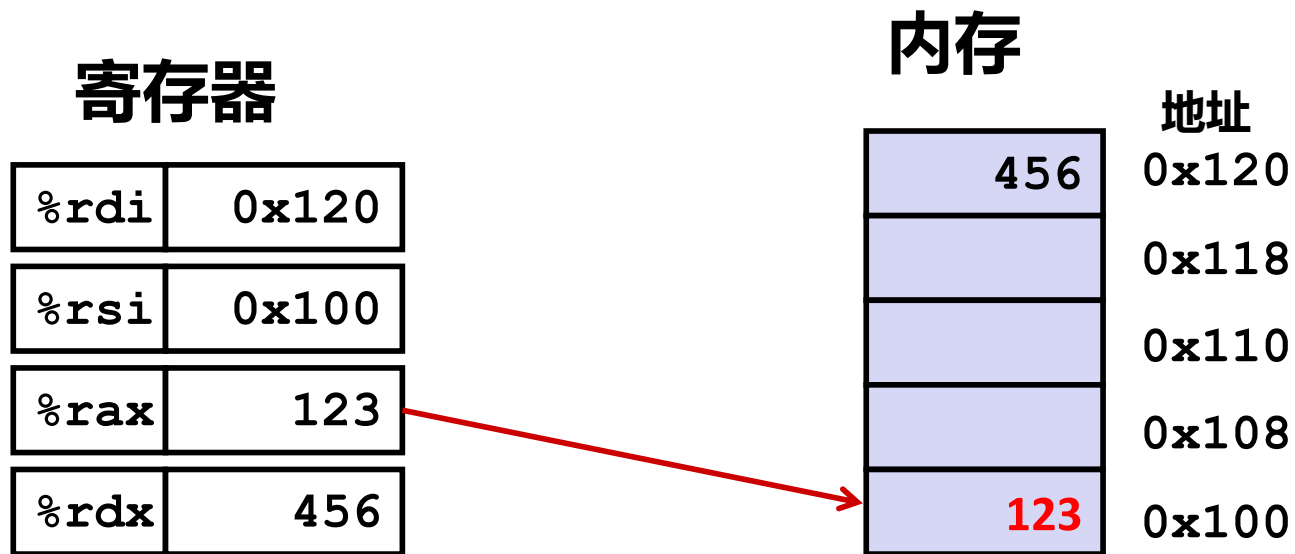
理解 Swap()



swap:

```
movq    (%rdi), %rax    # t0 = *xp
movq    (%rsi), %rdx    # t1 = *yp
movq    %rdx, (%rdi)    # *xp = t1
movq    %rax, (%rsi)    # *yp = t0
ret
```


理解 Swap()



swap:

```
movq    (%rdi), %rax    # t0 = *xp
movq    (%rsi), %rdx    # t1 = *yp
movq    %rdx, (%rdi)    # *xp = t1
movq    %rax, (%rsi)    # *yp = t0
ret
```

完整寻址模式

■ 最通用的形式

$D(Rb, Ri, S)$

$Mem[Reg[Rb] + S * Reg[Ri] + D]$

- D: 常偏移量, 1、2或4字节
- Rb: 基址寄存器, 16 个整数寄存器中的任何一个
- Ri: 变址寄存器, 除%rsp之外的任何寄存器
- S: 比例因子: 1, 2, 4, or 8 (为什么是这些数?)

■ 特殊情况

(Rb, Ri)

$Mem[Reg[Rb] + Reg[Ri]]$

$D(Rb, Ri)$

$Mem[Reg[Rb] + Reg[Ri] + D]$

(Rb, Ri, S)

$Mem[Reg[Rb] + S * Reg[Ri]]$

地址计算示例

<code>%rdx</code>	<code>0xf000</code>
<code>%rcx</code>	<code>0x0100</code>

D(Rb,Ri,S)

Mem[Reg[Rb]+S*Reg[Ri]+ D]

- D: 恒定“位移” 1、2或4字节
- Rb: 基址寄存器：16 个整数寄存器中的任何一个
- Ri: 变址寄存器：除`%rsp`之外的任何寄存器
- S: 比例因子: 1, 2, 4, or 8 (为什么是这些数?)

Expression	Address Computation	Address
<code>0x8(%rdx)</code>	<code>0xf000 + 0x8</code>	<code>0xf008</code>
<code>(%rdx,%rcx)</code>	<code>0xf000 + 0x100</code>	<code>0xf100</code>
<code>(%rdx,%rcx,4)</code>	<code>0xf000 + 4*0x100</code>	<code>0xf400</code>
<code>0x80(,%rdx,2)</code>	<code>2*0xf000 + 0x80</code>	<code>0x1e080</code>

简单的数据传送指令

Instruction		Effect	Description
MOV	S, D	$D \leftarrow S$	Move
movb			Move byte
movw			Move word
movl			Move double word
movq			Move quad word
movabsq	I, R	$R \leftarrow I$	Move absolute quad word

零扩展 数据传送指令

Instruction	Effect	Description
MOVZ S, R	$ R \leftarrow \text{ZeroExtend}(S)$	Move with zero extension
movzbw		Move zero-extended byte to word
movzbl		Move zero-extended byte to double word
movzwl		Move zero-extended word to double word
movzbq		Move zero-extended byte to quad word
movzwq		Move zero-extended word to quad word

符号扩展 数据传送指令

Instruction	Effect	Description
MOVS S, R	$R \leftarrow \text{SignExtend}(S)$	Move with sign extension
movsbw		Move sign-extended byte to word
movsbl		Move sign-extended byte to double word
movswl		Move sign-extended word to double word
movsbq		Move sign-extended byte to quad word
movswq		Move sign-extended word to quad word
movslq		Move sign-extended double word to quad word
cltq	$\%rax \leftarrow \text{SignExtend}(\%eax)$	Sign-extend %eax to %rax

练习 1

```
movabsq $0x0011223344556677, %rax
movb $0xAA, %dl
movb %dl,%al
movsbq %dl,%rax
movzbq %dl,%rax
```

%rax			
0x0011	2233	4455	6677
0x0011	2233	4455	6677
0x0011	2233	4455	66AA
0xFFFF	FFFF	FFFF	FFAA
0x0000	0000	0000	00AA

`%dl = AA`

练习 2

movb \$0xF, (%ebx)

✗

不能使用%ebx 作为地址寄存器

movl %rax, (%rsp)

✗

指令后缀和寄存器ID 不匹配

movw (%rax),4(%rsp)

✗

源操作数和目标操作数不能都是内存引用

movb %al,%sl

✗

没有寄存器叫做%sl

movq %rax,\$0x123

✗

立即数不能作为目的操作数

movl %eax,%rdx

✗

目标操作数大小不正确

movb %si, 8(%rbp)

✗

指令后缀和寄存器ID 不匹配

本节内容

- Intel 处理器的历史与架构
- C 代码，汇编代码，机器级代码
- **汇编语言基础：寄存器、操作数、move数据传送指令**
- 算术和逻辑操作

计算地址指令

■ `leaq Src, Dst`

- `src` 是寻址模式表达式
- `Dst` 目的地址表达式

■ 用法

- 在没有内存引用的情况下计算地址
 - 例如, `p = &x[i]` 的转换;
- 计算形如 `x + k*y` 的算术表达式
 - `k = 1, 2, 4, 8`

■ 例如

```
long m12(long x)
{
    return x*12;
}
```

编译器转换为汇编语言

```
leaq (%rdi,%rdi,2), %rax    # t = x+2*x
salq $2, %rax               # return t<<2
```

练习3

表达式

```
leaq 7(%rax, %rax, 8), %rdx
```

```
leaq 0xA(, %rcx, 4), %rdx
```

```
leaq 9(%rax, %rcx, 2), %rdx
```

结果

```
%rax = x
```

```
%rcx = y
```

练习 4

```
short scale3(short x, short y, short z) {  
    short t = _____;  
    return t;  
}
```

```
short scale3(short x, short y, short z)  
x in %rdi, y in %rsi, z in %rdx  
scale3:  
    leaq (%rsi,%rsi,9), %rbx  
    leaq (%rbx,%rdx), %rbx  
    leaq (%rbx,%rdi,%rsi), %rbx  
    ret
```

一些算术操作

■ 二元操作指令：

格式

计算

addq	Src, Dest	$\text{Dest} = \text{Dest} + \text{Src}$
subq	Src, Dest	$\text{Dest} = \text{Dest} - \text{Src}$
imulq	Src, Dest	$\text{Dest} = \text{Dest} * \text{Src}$
shlq	Src, Dest	$\text{Dest} = \text{Dest} \ll \text{Src}$
sarq	Src, Dest	$\text{Dest} = \text{Dest} \gg \text{Src}$
shrq	Src, Dest	$\text{Dest} = \text{Dest} \gg \text{Src}$
xorq	Src, Dest	$\text{Dest} = \text{Dest} \wedge \text{Src}$
andq	Src, Dest	$\text{Dest} = \text{Dest} \& \text{Src}$
orq	Src, Dest	$\text{Dest} = \text{Dest} \text{Src}$

代名词: salq

算术

逻辑

■ 注意参数的顺序！ *Src, Dest*

■ 有符号和无符号 int 之间没有区别（为什么？）

一些算术操作

■ 一元操作指令

<code>incq</code>	<code>Dest</code>	$\text{Dest} = \text{Dest} + 1$
<code>decq</code>	<code>Dest</code>	$\text{Dest} = \text{Dest} - 1$
<code>negq</code>	<code>Dest</code>	$\text{Dest} = -\text{Dest}$
<code>notq</code>	<code>Dest</code>	$\text{Dest} = \sim\text{Dest}$

■ 欲了解更多指令，请看课本

- 根据计数方式，总共有 2,034 条 x86 指令
- 如果算上所有的 `addr` 方式、操作宽度、标志，实际上是 3,683 条)

算术操作的表达式示例

```
long arith
(long x, long y, long z)
{
    long t1 = x+y;
    long t2 = z+t1;
    long t3 = x+4;
    long t4 = y * 48;
    long t5 = t3 + t4;
    long rval = t2 * t5;
    return rval;
}
```

arith:

```
leaq    (%rdi,%rsi), %rax
addq    %rdx, %rax
leaq    (%rsi,%rsi,2), %rdx
salq    $4, %rdx
leaq    4(%rdi,%rdx), %rcx
imulq    %rcx, %rax
ret
```

有趣的指令

- leaq: 地址计算
- salq: 移位
- imulq: 乘法
 - Curious: only used once...

理解算术表达式示例

```
long arith
(long x, long y, long z)
{
    long t1 = x+y;
    long t2 = z+t1;
    long t3 = x+4;
    long t4 = y * 48;
    long t5 = t3 + t4;
    long rval = t2 * t5;
    return rval;
}
```

arith:

```
leaq    (%rdi,%rsi), %rax    # t1
addq    %rdx, %rax          # t2
leaq    (%rsi,%rsi,2), %rdx
salq    $4, %rdx            # t4
leaq    4(%rdi,%rdx), %rcx   # t5
imulq   %rcx, %rax          # rval
ret
```

Register	Use(s)
%rdi	Argument x
%rsi	Argument y
%rdx	Argument z , t4
%rax	t1 , t2 , rval
%rcx	t5

程序的机器级表示 I：总结

■ Intel 处理器的历史与架构

- 不断演进的设计产生了很多奇怪之处

■ C 代码、汇编代码、机器代码

- 新的可见状态：程序计数器、寄存器.....
- 编译器需要将语句、表达式、过程转换为低级的指令序列

■ 汇编基础：寄存器、操作数、move数据传送指令

- x86-64 的move指令涵盖了各类型的数据传送形式

■ 算术和逻辑操作

- C 编译器会分辨出不同的指令组合，并执行计算